TASK 11: DATABASE AUTOMATION USING VIEWS AND PROCEDURES

PROJECT DETAILS

- **Name:** Kaviya S
- **Roll No:** E25AI118
- **Department:** B.Sc Computer Science with AI

SCENARIO

The bookstore **PALETTE: Pixel & Pages** frequently generates reports and performs repetitive database operations such as order processing, sales calculation, inventory management, and customer analysis. To improve efficiency and reduce repeated SQL coding, the bookstore wants to automate database operations using Views, Stored Procedures, and Functions. Students will implement database automation features in the existing Bookstore Database System.

---

1. CREATE VIEWS FOR FREQUENTLY USED REPORTS

Concept

A View is a virtual table created from one or more SQL queries. It stores the query definition and provides simplified access to frequently used information.

1.1 Customer Order Report View

Display:

- Customer Name
- Order ID
- Order Date
- Total Amount
- Order Status

SQL Script

```
SQL> CREATE OR REPLACE VIEW CUSTOMER_ORDER_REPORT AS
  2  SELECT
  3  C.FIRST_NAME || ' ' || C.LAST_NAME AS CUSTOMER_NAME,
  4  O.ORDER_ID,
  5  O.ORDER_DATE,
  6  O.TOTAL_AMOUNT,
  7  O.ORDER_STATUS
  8  FROM CUSTOMER C
  9  JOIN ORDERS O
 10  ON C.CUSTOMER_ID = O.CUSTOMER_ID;

View created.
```

Output

```
SQL>  SELECT * FROM CUSTOMER_ORDER_REPORT;

CUSTOMER_NAME                          ORDER_ID     ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
-------------------------------------- ------------ --------- ------------ --------------------
Ananya Roy                             ORD-501      01-MAR-26         1098 Delivered
Ananya Roy                             ORD-502      05-MAR-26         1250 Delivered
Rohan Mehta                            ORD-503      10-MAR-26          599 Pending
Priya Nair                             ORD-504      12-MAR-26         1149 Shipped
Aarav Sharma                           ORD-505      15-MAR-26          699 Delivered
Ishita Menon                           ORD-506      16-MAR-26          799 Pending
Kabir Singh                            ORD-507      17-MAR-26          499 Shipped
Diya Patel                             ORD-508      18-MAR-26          599 Delivered
Arjun Nair                             ORD-509      19-MAR-26          450 Delivered
Meera Rao                              ORD-510      20-MAR-26          650 Pending
Rohan Verma                            ORD-511      21-MAR-26         1999 Delivered
Anika Iyer                             ORD-512      22-MAR-26         1799 Shipped
Nikhil Bose                            ORD-513      23-MAR-26          550 Pending
Pooja Reddy                            ORD-514      24-MAR-26          799 Delivered
Vikram Kapoor                          ORD-515      25-MAR-26          899 Delivered
Sneha Gupta                            ORD-516      26-MAR-26          450 Cancelled
Aditya Mishra                          ORD-517      27-MAR-26         1599 Delivered
Kavya Das                              ORD-518      28-MAR-26          750 Shipped
Rahul Joseph                           ORD-519      29-MAR-26          399 Pending
Neha Varma                             ORD-520      30-MAR-26          550 Delivered
```

1.2 Product Sales Report View (Books)

Display:

- Product Name (Book Title)
- Quantity Sold
- Total Revenue

SQL Script

```
SQL> CREATE OR REPLACE VIEW PRODUCT_SALES_REPORT AS
  2  SELECT
  3  B.TITLE AS PRODUCT_NAME,
  4  SUM(OD.QUANTITY) AS QUANTITY_SOLD,
  5  SUM(OD.QUANTITY * OD.PRICE) AS TOTAL_REVENUE
  6  FROM BOOK B
  7  JOIN ORDER_DETAILS OD
  8  ON B.BOOK_ID = OD.PRODUCT_ID
  9  GROUP BY B.TITLE;

View created.
```

Output

```
SQL> SELECT * FROM PRODUCT_SALES_REPORT;

PRODUCT_NAME                              QUANTITY_SOLD TOTAL_REVENUE
----------------------------------------- ------------- -------------
Gone Girl                                             1           799
Harry Potter and the Philosopher's Stone              1           999
Dracula                                               1           550
The Inheritance Games                                 3          1899
Dune                                                  2          1198
The Silent Patient                                    1           699
Sherlock Holmes                                       1           499
Pride and Prejudice                                   1           450
The Alchemist                                         2           949
Murder on the Orient Express                          1           599
The Love Hypothesis                                   1           899
Foundation                                            1           650
Designing Data-Intensive Applications                 1          1250
The Great Gatsby                                      2           998
Me Before You                                         1           650
The Hobbit                                            1           850
The Shining                                           1           799

17 rows selected.
```

1.3 Payment Status View

Display:

- Order ID
- Payment Mode
- Payment Status
- Transaction Amount

SQL Script

```
SQL> CREATE OR REPLACE VIEW PAYMENT_STATUS_VIEW AS
  2  SELECT
  3         ORDER_ID,
  4         PAYMENT_MODE,
  5         PAYMENT_STATUS,
  6         TRANSACTION_AMOUNT
  7  FROM PAYMENT;

View created.
```

Output

```
SQL> SELECT * FROM PAYMENT_STATUS_VIEW;

ORDER_ID     PAYMENT_MODE    PAYMENT_STATUS  TRANSACTION_AMOUNT
------------ --------------- --------------- ------------------
ORD-501      Credit Card     Completed                 1098.00
ORD-502      UPI             Completed                 1250.00
ORD-503      Debit Card      Pending                    599.00
ORD-504      Net Banking     Completed                 1149.00
ORD-505      UPI             Completed                  699.00
ORD-506      Credit Card     Pending                    799.00
ORD-507      UPI             Completed                  499.00
ORD-508      Debit Card      Completed                  599.00
ORD-509      Net Banking     Completed                  450.00
ORD-510      UPI             Pending                    650.00
ORD-511      Credit Card     Completed                  999.00
ORD-512      Debit Card      Completed                  850.00
ORD-513      UPI             Pending                    550.00
ORD-514      Credit Card     Completed                  799.00
ORD-515      Net Banking     Completed                  899.00
ORD-516      UPI             Failed                     450.00
ORD-517      Credit Card     Completed                  699.00
ORD-518      Debit Card      Completed                  750.00
ORD-519      UPI             Pending                    399.00
ORD-520      Net Banking     Completed                  550.00

20 rows selected.
```

---

2. DEVELOP STORED PROCEDURES FOR ORDER PROCESSING

Concept

A Stored Procedure is a collection of SQL statements stored in the database that can be executed whenever required. It helps automate repeated operations.

2.1 Add New Customer Order

Process:

- Insert order details
- Add purchased products
- Calculate total amount

SQL Script

```
SQL> CREATE OR REPLACE PROCEDURE ADD_NEW_ORDER
  2  (
  3  P_ORDER_ID IN VARCHAR2,
  4  P_CUSTOMER_ID IN VARCHAR2,
  5  P_TOTAL_AMOUNT IN NUMBER
  6  )
  7  IS
  8  BEGIN
  9
 10  INSERT INTO ORDERS
 11  (
 12  ORDER_ID,
 13  CUSTOMER_ID,
 14  ORDER_DATE,
 15  TOTAL_AMOUNT,
 16  ORDER_STATUS
 17  )
 18  VALUES
 19  (
 20  P_ORDER_ID,
 21  P_CUSTOMER_ID,
 22  SYSDATE,
 23  P_TOTAL_AMOUNT,
 24  'Pending'
 25  );
 26
 27  COMMIT;
 28
 29  END;
 30  /

Procedure created.
```

Procedure Execution

```
SQL> EXEC ADD_NEW_ORDER('ORD-521', 'CUST-120', 650);

PL/SQL procedure successfully completed.
```

2.2 Update Order Status

Example:

Pending → Shipped → Delivered

SQL Script

```
SQL> CREATE OR REPLACE PROCEDURE UPDATE_ORDER_STATUS
  2  (
  3  P_ORDER_ID IN VARCHAR2,
  4  P_STATUS IN VARCHAR2
  5  )
  6  IS
  7  BEGIN
  8
  9  UPDATE ORDERS
 10  SET ORDER_STATUS = P_STATUS
 11  WHERE ORDER_ID = P_ORDER_ID;
 12
 13  COMMIT;
 14
 15  END;
 16  /

Procedure created.
```

Procedure Execution

```
SQL> EXEC UPDATE_ORDER_STATUS('ORD-521', 'Delivered');

PL/SQL procedure successfully completed.
```

2.3 Generate Customer Purchase History

Input:

- Customer_ID

Output:

- Order Details
- Purchase Amount
- Order Status

SQL Script

```
SQL> CREATE OR REPLACE PROCEDURE GET_CUSTOMER_ORDERS
  2  (
  3  P_CUSTOMER_ID IN VARCHAR2
  4  )
  5  IS
  6  BEGIN
  7
  8  FOR REC IN
  9  (
 10  SELECT
 11  ORDER_ID,
 12  TOTAL_AMOUNT,
 13  ORDER_STATUS
 14  FROM ORDERS
 15  WHERE CUSTOMER_ID = P_CUSTOMER_ID
 16  )
 17  LOOP
 18
 19  DBMS_OUTPUT.PUT_LINE
 20  (
 21  REC.ORDER_ID || ' ' ||
 22  REC.TOTAL_AMOUNT || ' ' ||
 23  REC.ORDER_STATUS
 24  );
 25
 26  END LOOP;
 27
 28  END;
 29  /
```

Procedure Execution

```
SQL> EXEC GET_CUSTOMER_ORDERS('CUST-101');
ORD-501 1098 Delivered
ORD-502 1250 Delivered

PL/SQL procedure successfully completed.
```

---

3. CREATE FUNCTIONS FOR SALES CALCULATIONS

Concept

A Function performs a specific calculation and returns a value.

3.1 Calculate Total Sales

Input:

- Date Range

Output:

- Total Revenue Generated

SQL Script

```
SQL> CREATE OR REPLACE FUNCTION CALCULATE_TOTAL_SALES
  2  (
  3  P_START_DATE DATE,
  4  P_END_DATE DATE
  5  )
  6  RETURN NUMBER
  7  IS
  8
  9  V_TOTAL NUMBER;
 10
 11  BEGIN
 12
 13  SELECT NVL(SUM(TOTAL_AMOUNT),0)
 14  INTO V_TOTAL
 15  FROM ORDERS
 16  WHERE ORDER_DATE
 17  BETWEEN P_START_DATE
 18  AND P_END_DATE;
 19
 20  RETURN V_TOTAL;
 21
 22  END;
 23  /

Function created.
```

Function Output

```
SQL> SELECT CALCULATE_TOTAL_SALES
  2  (
  3  TO_DATE('01-JAN-26','DD-MON-RR'),
  4  SYSDATE
  5  )
  6  AS TOTAL_SALES
  7  FROM DUAL;

TOTAL_SALES
-----------
      18836
```

3.2 Calculate Customer Total Spending

Input:

- Customer_ID

Output:

- Total Amount Spent

SQL Script

```
SQL> CREATE OR REPLACE FUNCTION CUSTOMER_TOTAL_SPENDING
  2  (
  3  P_CUSTOMER_ID VARCHAR2
  4  )
  5  RETURN NUMBER
  6  IS
  7
  8  V_TOTAL NUMBER;
  9
 10  BEGIN
 11
 12  SELECT NVL(SUM(TOTAL_AMOUNT),0)
 13  INTO V_TOTAL
 14  FROM ORDERS
 15  WHERE CUSTOMER_ID = P_CUSTOMER_ID;
 16
 17  RETURN V_TOTAL;
 18
 19  END;
 20  /

Function created.
```

Function Output

```
SQL> SELECT CUSTOMER_TOTAL_SPENDING('CUST-101')
  2  FROM DUAL;

CUSTOMER_TOTAL_SPENDING('CUST-101')
-----------------------------------
                               2348
```

3.3 Calculate Product Revenue (Book)

Input:

- Product_ID (Book_ID)

Output:

- Total Sales Amount

SQL Script

```
SQL> CREATE OR REPLACE FUNCTION PRODUCT_REVENUE
  2  (
  3  P_PRODUCT_ID VARCHAR2
  4  )
  5  RETURN NUMBER
  6  IS
  7
  8  V_REVENUE NUMBER;
  9
 10  BEGIN
 11
 12  SELECT NVL
 13  (
 14  SUM(QUANTITY * PRICE),
 15  0
 16  )
 17  INTO V_REVENUE
 18  FROM ORDER_DETAILS
 19  WHERE PRODUCT_ID = P_PRODUCT_ID;
 20
 21  RETURN V_REVENUE;
 22
 23  END;
 24  /

Function created.
```

Function Output

```
SQL> SELECT PRODUCT_REVENUE('B1025')
  2  FROM DUAL;

PRODUCT_REVENUE('B1025')
------------------------
                    1899
```

---

4. AUTOMATE REPETITIVE DATABASE OPERATIONS

4.1 Order Processing Automation

When a customer places an order:

1. Insert order details.
2. Reduce product stock.
3. Create payment record.
4. Update order status.

SQL Script

```
SQL> CREATE OR REPLACE PROCEDURE PROCESS_ORDER
  2  (
  3  P_ORDER_ID VARCHAR2,
  4  P_PRODUCT_ID VARCHAR2,
  5  P_QUANTITY NUMBER
  6  )
  7  IS
  8  BEGIN
  9
 10  UPDATE INVENTORY
 11  SET STOCK_QUANTITY =
 12  STOCK_QUANTITY - P_QUANTITY
 13  WHERE PRODUCT_ID = P_PRODUCT_ID;
 14
 15  UPDATE ORDERS
 16  SET ORDER_STATUS = 'Processed'
 17  WHERE ORDER_ID = P_ORDER_ID;
 18
 19  COMMIT;
 20
 21  END;
 22  /

Procedure created.
```

4.2 Inventory Update Automation

Automatically:

- Increase stock after new supply.
- Reduce stock after purchase.
- Identify low-stock products.

SQL Script

```
SQL> CREATE OR REPLACE VIEW LOW_STOCK_PRODUCTS AS
  2  SELECT
  3  PRODUCT_ID,
  4  STOCK_QUANTITY,
  5  STOCK_STATUS
  6  FROM INVENTORY
  7  WHERE STOCK_QUANTITY < 10;

View created.
```

Output

```
SQL> SELECT * FROM LOW_STOCK_PRODUCTS;

PRODUCT STOCK_QUANTITY STOCK_STATUS
------- -------------- --------------------
B1002                8 Available
B1014                9 Available
```

4.3 Report Generation Automation

Automatically Generate:

- Daily Sales Report
- Monthly Revenue Report
- Customer Purchase Report

Daily Sales Report

```
SQL> SELECT
  2  TRUNC(ORDER_DATE) AS SALES_DATE,
  3  SUM(TOTAL_AMOUNT) AS DAILY_SALES
  4  FROM ORDERS
  5  GROUP BY TRUNC(ORDER_DATE);

SALES_DAT DAILY_SALES
--------- -----------
12-MAR-26        1149
16-MAR-26         799
18-MAR-26         599
23-MAR-26         550
25-MAR-26         899
08-NOV-26        1250
05-MAR-26        1250
15-MAR-26         699
22-MAR-26        1799
26-MAR-26         450
28-MAR-26         750
10-MAR-26         599
21-MAR-26        1999
24-MAR-26         799
29-MAR-26         399
01-MAR-26        1098
27-MAR-26        1599
30-MAR-26         550
17-MAR-26         499
19-MAR-26         450
20-MAR-26         650

21 rows selected.
```

Monthly Revenue Report

```
SQL> SELECT
  2  TO_CHAR(ORDER_DATE,'MON-YYYY') AS MONTH,
  3  SUM(TOTAL_AMOUNT) AS REVENUE
  4  FROM ORDERS
  5  GROUP BY TO_CHAR(ORDER_DATE,'MON-YYYY');

MONTH                REVENUE
---------------- ----------
MAR-2026              17586
NOV-2026               1250
```

Customer Purchase Report

```
SQL> SELECT
  2  CUSTOMER_ID,
  3  SUM(TOTAL_AMOUNT) AS TOTAL_PURCHASE
  4  FROM ORDERS
  5  GROUP BY CUSTOMER_ID;

CUSTOMER_ID          TOTAL_PURCHASE
-------------------- --------------
CUST-102                        599
CUST-108                        599
CUST-109                        450
CUST-112                       1799
CUST-119                        399
CUST-115                        899
CUST-116                        450
CUST-107                        499
CUST-101                       2348
CUST-106                        799
CUST-110                        650
CUST-113                        550
CUST-118                        750
CUST-105                        699
CUST-120                       1800
CUST-114                        799
CUST-103                       1149
CUST-111                       1999
CUST-117                       1599

19 rows selected.
```

---

5. TEST DATABASE OBJECTS

Views

Check:

- Correct Data Retrieval
- Report Accuracy

Testing Customer Order Report View

```
SQL> SELECT * FROM CUSTOMER_ORDER_REPORT;

CUSTOMER_NAME                          ORDER_ID     ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
-------------------------------------- ------------ --------- ------------ --------------------
Ananya Roy                             ORD-501      01-MAR-26         1098 Delivered
Ananya Roy                             ORD-502      05-MAR-26         1250 Delivered
Rohan Mehta                            ORD-503      10-MAR-26          599 Pending
Priya Nair                             ORD-504      12-MAR-26         1149 Shipped
Aarav Sharma                           ORD-505      15-MAR-26          699 Delivered
Ishita Menon                           ORD-506      16-MAR-26          799 Pending
Kabir Singh                            ORD-507      17-MAR-26          499 Shipped
Diya Patel                             ORD-508      18-MAR-26          599 Delivered
Arjun Nair                             ORD-509      19-MAR-26          450 Delivered
Meera Rao                              ORD-510      20-MAR-26          650 Pending
Rohan Verma                            ORD-511      21-MAR-26         1999 Delivered
Anika Iyer                             ORD-512      22-MAR-26         1799 Shipped
Nikhil Bose                            ORD-513      23-MAR-26          550 Pending
Pooja Reddy                            ORD-514      24-MAR-26          799 Delivered
Vikram Kapoor                          ORD-515      25-MAR-26          899 Delivered
Sneha Gupta                            ORD-516      26-MAR-26          450 Cancelled
Aditya Mishra                          ORD-517      27-MAR-26         1599 Delivered
Kavya Das                              ORD-518      28-MAR-26          750 Shipped
Rahul Joseph                           ORD-519      29-MAR-26          399 Pending
Neha Varma                             ORD-520      30-MAR-26          550 Delivered
Neha Varma                             ORD-521      08-NOV-26         1250 Delivered

21 rows selected.
```

Testing Product Sales Report View (Books)

```
SQL> SELECT * FROM PRODUCT_SALES_REPORT;

PRODUCT_NAME                                QUANTITY_SOLD TOTAL_REVENUE
------------------------------------------- ------------- -------------
Gone Girl                                               1           799
Harry Potter and the Philosopher's Stone                1           999
Dracula                                                 1           550
The Inheritance Games                                   3          1899
Dune                                                    2          1198
The Silent Patient                                      1           699
Sherlock Holmes                                         1           499
Pride and Prejudice                                     1           450
The Alchemist                                           2           949
Murder on the Orient Express                            1           599
The Love Hypothesis                                     1           899
Foundation                                              1           650
Designing Data-Intensive Applications                   1          1250
The Great Gatsby                                        2           998
Me Before You                                           1           650
The Hobbit                                              1           850
The Shining                                             1           799

17 rows selected.
```

Testing Payment Status View

```
SQL> SELECT * FROM PAYMENT_STATUS_VIEW;

ORDER_ID     PAYMENT_MODE    PAYMENT_STATUS  TRANSACTION_AMOUNT
------------ --------------- --------------- ------------------
ORD-501      Credit Card     Completed                 1098.00
ORD-502      UPI             Completed                 1250.00
ORD-503      Debit Card      Pending                    599.00
ORD-504      Net Banking     Completed                 1149.00
ORD-505      UPI             Completed                  699.00
ORD-506      Credit Card     Pending                    799.00
ORD-507      UPI             Completed                  499.00
ORD-508      Debit Card      Completed                  599.00
ORD-509      Net Banking     Completed                  450.00
ORD-510      UPI             Pending                    650.00
ORD-511      Credit Card     Completed                  999.00
ORD-512      Debit Card      Completed                  850.00
ORD-513      UPI             Pending                    550.00
ORD-514      Credit Card     Completed                  799.00
ORD-515      Net Banking     Completed                  899.00
ORD-516      UPI             Failed                     450.00
ORD-517      Credit Card     Completed                  699.00
ORD-518      Debit Card      Completed                  750.00
ORD-519      UPI             Pending                    399.00
ORD-520      Net Banking     Completed                  550.00
ORD-521      UPI             Completed                 1250.00

21 rows selected.
```

Procedures

Test:

- Procedure Execution
- Input Parameters
- Output Results

Testing

- Add New Customer Order Procedure
- Update Order Status Procedure
- Customer Purchase History Procedure

```
SQL> EXEC ADD_NEW_ORDER('ORD-522', 'CUST-102', 899);

PL/SQL procedure successfully completed.

SQL> EXEC UPDATE_ORDER_STATUS('ORD-522', 'Delivered');

PL/SQL procedure successfully completed.

SQL> EXEC GET_CUSTOMER_ORDERS('CUST-102');
ORD-503 599 Pending
ORD-522 899 Delivered

PL/SQL procedure successfully completed.
```

Functions

Verify:

- Calculation Accuracy
- Return Values

Testing:

- Total Sales Function
- Customer Total Spending Function
- Product Revenue Function (Book)

```
SQL> SELECT CUSTOMER_TOTAL_SPENDING('CUST-101')
  2  FROM DUAL;

CUSTOMER_TOTAL_SPENDING('CUST-101')
-----------------------------------
                               2348

SQL>
SQL> SELECT PRODUCT_REVENUE('B1025')
  2  FROM DUAL;

PRODUCT_REVENUE('B1025')
------------------------
                    1899

SQL>
SQL> SELECT CALCULATE_TOTAL_SALES
  2  (
  3  TO_DATE('01-JAN-26','DD-MON-RR'),
  4  SYSDATE
  5  )
  6  FROM DUAL;

CALCULATE_TOTAL_SALES(TO_DATE('01-JAN-26','DD-MON-RR'),SYSDATE)
---------------------------------------------------------------
                                                          19735
```

CONCLUSION

The Database Automation System successfully implements Views, Stored Procedures, and Functions to automate repetitive database operations in the bookstore. The automation improves report generation, order processing, customer analysis, inventory monitoring, and sales calculations while reducing repetitive SQL coding and improving database efficiency.